

Module avansate pentru sisteme în timp real
Librăriile TCP-IP de la Silabs

Obiectiv

În acest laborator vom prezenta un exemplu de module realizate pentru sistemele în timp real. Acestea sunt modulele TCP-IP realizate de firma Silabs cei care au realizat și microcontrolerul C8051F120. Librăriile sunt destinate implementării stivei TCP-IP în microsisteme din familia C8051f1xx pentru realizarea de aplicații care să furnizeze servicii tipice acestui protocol. În laborator se va implementa un server HTTP care va rula ca un task distinct ce poate fi folosit în paralel cu alte procese.

Breviar

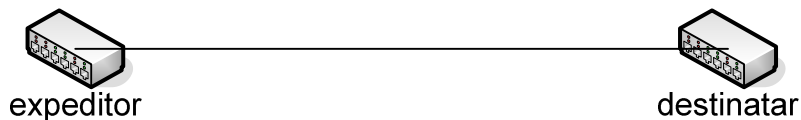
În cele ce urmează se va ilustra folosirea unor librării avansate, create de cei de la Silabs pentru utilizarea serviciilor de rețea în aplicații cu microsisteme.

Pentru deplina înțelegere a noțiunilor introduse în laborator vom face o scurtă prezentare a serviciilor de rețea.

Protocolul TCP-IP și serviciile

Comunicația pe interfață serială Ethernet este una extrem de folosită la ora actuală. Practic orice calculator conectat la internet are de fapt o asemenea interfață de comunicație. Protocolul de comunicație prin intermediul acestor interfețe a devenit din ce în ce mai complex pentru a face față cerințelor de securitate și protecție a datelor.

Pentru a fi mai explicit să considerăm următorul exemplu: o comunicație serială între un expeditor și un destinatar.



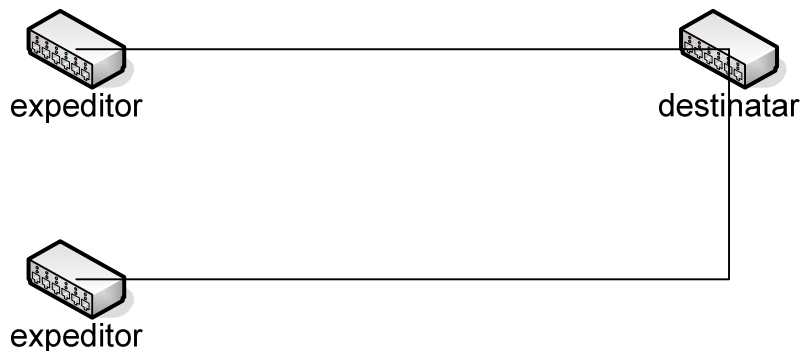
Presupunem că expeditorul dorește să transmită un mesaj format din 8 octeți destinatarului.

În primul rând tipul comunicației fiind serială (bit cu bit) este necesar un mijloc pentru a sincroniza expeditorul cu destinatarul pentru recepția unui bit. Din aceste motive fiecare octet din mesajul de transmis trebuie să aibă la început un bit de start iar la sfârșit unul de stop. Prin acești biți suplimentari informează destinatarul de momentele în care începe și se sfârșește un cuvânt. Deci, la fiecare octet în parte, alături de informația utilă care reprezintă ceea ce se dorește transmis se mai adaugă și elemente suplimentare cum ar fi biți de sincronizare (start și stop) necesari într-o comunicație serială. De asemenea se poate adăuga și un bit suplimentar prin care să se verifice corectitudinea șirului de biți transmis pentru un octet. Putem vorbi din acest moment despre un pachet de date serial care conține alături de informația de date și informații suplimentare care nu privesc datele dar care sunt strict necesare într-o comunicație serială. Singurul lucru de amintit este că

dimensiunea unui pachet serial nu poate fi prea mare (maxim 9 biți) din considerente legate de calitatea datelor transmise.

Pe de altă parte, se poate dori verificarea corectitudinii mesajului recepționat din punctul de vedere al succesiunii de recepție al octeților. Cu alte cuvinte dacă octeții din mesaj au ajuns sau nu în ordine. De asemenea se poate dori implementarea unui protocol suplimentar pentru verificarea corectitudinii octeților transmiși. Deci, alături de cei 8 octeți, care acum îi privim ca 8 pachete seriale, se poate adăuga și un al 9-lea octet - pachet care reprezintă suma de control.

O altă situație ar fi cea din figura de mai jos. Aici de imaginăm că există doi expeditori și un destinatar.



În această situație la cele 9 pachete seriale se adaugă un al 10-lea care reprezintă un cod al expeditorului eventual un al 11-lea pachet care reprezintă un octet de start al comunicației pentru sincronizarea cu destinatarul. În figura de mai jos sunt arătate toate aceste adăugări la informația de date utilă:

Pachet de date original	41h 42h 43h ... 48h	Datele care se doresc transmise
Pachet serial pentru fiecare octet în parte	0 0100 0001 01 => 41h 42h 43h ... 48h	Adăugarea bitului de start stop și paritate la fiecare octet în parte
Pachete seriale cu suma de control la nivel de octet adăugate	41h 42h 43h ... 48h 24h	Adăugarea sumei de control pentru determinarea corectitudinii la nivel de octeți
Pachete seriale cu octetul de start și octetul de adresă adăugate	aah 01h 41h 42h 43h ... 48h 24h	Adăugarea octetului de start și octetului de adresă în cazul unei comunicații cu mai mulți expeditori

Astfel, se poate imagina situația de mai sus ca un protocol cu mai multe nivele. Un nivel este cel care formează pachetele seriale iar altul cel care adăugă la nivel de octet.

În comunicația ethernet intervin mulți mai mulți factori. În primul rând, între un expeditor și un destinatar se pot afla mai multe stații intermediare, deci la un nivel se determină expeditorul și destinatarul temporal iar la alt nivel se determină expeditorul și

destinatarul final. Toate nivelele au propriile elemente care le adaugă la pachet, după o metodă similară cu cea prezentată în exemplul simplificat de mai sus.

Pentru standardizare s-au elaborat mai multe modele de protocoale toate plecând însă de la modelul OSI care are 7 straturi.

Modelul de Referință OSI (engl.: *Open Systems Interconnection - Reference Model*), pe scurt: OSI, al organizației *International Organization for Standardization*, numită și ISO, este o structură de **comunicare** ierarhică foarte des folosită într-o **rețea**. Modelul OSI folosește 7 așa-numite straturi pentru a transmite și primi **date** eficient și rapid. Fiecare strat are funcții clar definite. **Internetul** este bazat pe acest model.

Straturile modelului OSI și funcțiunile lor:

7. **Aplicație** (engl.: *Application Layer*) - Aplicația care rulează transmiterea și primirea datelor, de ex. **FTP**, **HTTP**

6. **Prezentare** (*Presentation Layer*) - formatează datele pentru a putea fi utilizate de către aplicație

5. **Sesiune** (*Session Layer*) - stabilește, gestionează și termină sesiunile între aplicații

4. **Transport** (*Transport Layer*) - se ocupă cu segmente de informație

3. **Rețea** (*Network Layer*) - pachete de informație

2. **Conexiunea datelor** (*Data Link Layer*) - asigură dirijarea adreselor fizice, topologiei rețelelor, notificarea erorilor și a cadrelor de biți.

1. **Fizic** (*Physical Layer*) - totalitatea de specificații electrice, mecanice, procedurale și funcționale pentru asigurarea conexiunii fizice a două sisteme.

(după wikipedia <http://ro.wikipedia.org/wiki/OSI>)

Fiecare strat are propriul său set de protocoale, adică ce anume se va adăuga la pachetul de date.

Exemple de protocoale din Stiva OSI

7	Aplicație	ex: HTTP , SMTP , SNMP , FTP , Telnet , SIP , SSH , NFS , RTSP , XMPP , Whois , ENRP
6	Prezentare	ex: XDR , ASN.1 , SMB , AFP , NCP
5	Sesiune	ex: ASAP , TLS , SSH , ISO 8327 / CCITT X.225, RPC , NetBIOS ,

		ASP, Winsock, BSD sockets
4	Transport	ex: TCP, UDP, RTP, SCTP, SPX, ATP, IL
3	Rețea	ex: IP, ICMP, IGMP, IPX, BGP, OSPF, RIP, IGRP, EIGRP, ARP, RARP, X.25
2	Legătura de date	ex: Ethernet, Token ring, HDLC, Frame relay, ISDN, ATM, 802.11 WiFi, FDDI, PPP
1	Fizic	ex: cablu coaxial, radio, fibră optică, cablu bifilar torsadat, fire cupru

(după wikipedia <http://ro.wikipedia.org/wiki/OSI>)

Numărul de protocoale (servicii) care le oferă comunicația ethernet este foarte mare iar prezentarea lor detaliată nu este obiectivul acestui laborator. Voi menționa doar că protocolul http este cel responsabil în comunicația informațiilor care sunt afișate într-un navigator de internet (Internet Explorer, Netscape, Opera etc.) adică partea „vizibilă” a internetului. Protocolul FTP se ocupă de transferul de fișiere iar protocolul SMTP de transferul e-mail-urilor.

Toate protocoalele enumerate mai sus urmăresc modul de lucru al unei transmisii seriale de tipul half duplex prezentate mai sus. Adică, la un moment dat, o stație nu poate fi decât expeditor iar alta destinatar. La nivelul protocoalelor de rețea aceasta s-ar traduce că există un server și un client. Calculatorul pe care este afișată o pagină de internet reprezintă un client http iar browser-ul aplicația client. Serverul http reprezintă calculatorul pe care se află stocată informația care apare într-o pagină pe browser.

La ora actuală s-au creat librării care permit încărcarea unor servere pe microsisteme. Astfel, pentru aplicații portabile sau care necesită un consum redus sau funcționare în condiții de mediu defavorabile calculatorul se va înlocui cu un microsistem pe care în prealabil a fost implementat un server. Problema care apare aici este următoarea, serverul trebuie să răspundă permanent cerințelor unui client. Din punct de vedere al unui sistem în timp real putem vedea serverul ca un proces care răspunde unor secvențe spontane. Este evident că un server poate fi concepută ca o aplicație tipică în timp real, deoarece critic la un server este timpul de răspuns la cerințele unui client. Din aceste motive aplicațiile server pe microsisteme sunt legate de aplicații în timp real.

Există mai multe librării care au implementate în ele protocoale de rețea (ZTP, DinamicC, Microchip TCPIP, Micrium TCPStack etc) iar ele sunt de regulă dedicate pentru anumite familii de microcontrolere. Și cei de la Silabs, fabricanții microcontrolerului C8051F120 au creat librării pentru protocoale de rețea pentru aceasta familie de microcontrolere.

Aceste librării suportă mai multe protocoale: DHCP, http, SMTP, TCP, UDP. Din acestea în cele ce urmează se va construi un server http care se va implementa pe platforma de laborator.

Bineînțeles, pentru accesul la protocoalele de comunicație în rețea nu este suficient numai setul de librării ci și interfața de comunicație fizică. Se observă că machetele de laborator sunt dotate cu interfețe ethernet.

Desfășurarea lucrării

Dezvoltarea unui server http implică mai multe etape.

- O etapă o reprezintă proiectarea site-ului – pagina sau paginile web care vor fi apoi accesate de client. Pentru proiectarea paginilor web se folosesc limbaje specializate printre care cel mai comun este limbajul HTML.
- A doua etapă constă în proiectarea serverului propriu-zis, capabil să răspundă la cerințele clientului.

Pentru ca proiectul să ruleze corect se va proceda astfel:

- se va crea pe discul D un director numit lab11;
- în interiorul directorului se va crea un subdirector denumit pagină.

Pe discul D se află un director care se numește pachetlab11. Acesta conține platforma de față și o serie de fișiere, este vorba de fișierele librării ale lui Silabs. Din acest director se vor lua următoarele fișiere și se vor plasa în directoarele în care se lucrează:

Fișier	Director destinație
Mn_callback.c	Lab11
Mn_vars.c	Lab11
Mn_defs.h	Lab11
Mn_errs.h	Lab11
Mn_funcs.h	Lab11
Mn_stackconst.h	Lab11
Mn_userconst.h	Lab11
Mn_stack_bank_039.lib	Lab11
Mn_stack_common_039.lib	Lab11
Mn_defs.h	Lab11\Pagina
Mn_stackconst.h	Lab11\Pagina
Html2c.exe	Lab11\Pagina

Fișierul html2c.exe este un utilitar care va fi folosit pentru conversia paginilor create din format html, în care ele sunt descrise, în formatul c pentru a putea fi apoi importate în proiect.

Etapa 1 Crearea site-ului (pagina web)

Pentru crearea site-ului este necesară utilizarea unor limbaje specializate. Cel mai comun dintre acestea este html. Pentru crearea unui fișier HTML se procedează astfel:

- se deschide aplicația Notepad din Windows;
- în interior se scrie următorul text:

```
<html>
```

```
<head><title>Laborator sisteme in timp real</title><head>
```

```
<body bgcolor="green" text="white" link="yellow" vlink="red" alink="blue">
```

```

<h1>Laboratorul 11</h1> <br><br>
<B>Laborator cu librarii TCPIP pentru C8051F120</B><br>
La multi ani si mult succes in 2007!!!
</body>
</html>

```

- se salvează fișierul sub numele de index.html în directorul Lab11\Pagina.

Site-ul nostru va conține doar o pagină numită index.html. Se pot construi site-uri mai elaborate, care pot conține link-uri, imagini etc.

După salvare, fișierul are în mod normal atașat ca program implicit de rulare browser-ul activ. Pentru a verifica corectitudinea introducerii datelor se va rula fișierul și se va observa pagina pe calculator.

Etapa următoare constă în conversia fișierului din format html în format c și h pentru a putea fi importat în proiectul serverului. În urma acestei conversii, fișierul html este transformat într-un șir de octeți (coduri ASCII) care sunt apoi transmise de server la client. Practic interpretarea este făcută de către browser. Se vor crea de către utilitar două fișiere index.c și index.h.

Etapa 2 crearea serverului

Se va crea un proiect în mediul SilabsIDE care va fi salvat în directorul Lab11.

Se va crea un fișier c main.c în care se vor scrie următoarele:

```

//-----
// main.c
//-----
// Etape pentru rularea acestui program:
//
// 1. Se inlocuieste IP_SRC_ADDR din mn_userconst.h cu adresa IP dorita
//
// 2. Se conecteaza la C8051F120
//
// 3. Se compileaza si construiesc proiectul
//
// 4. Se descarca codul in platforma si se ruleaza programul
//
// 5. Se deschide un browser si se conecteaza la http:\\XXX.XXX.XXX.XXX"
//    in bara cu adrese (unde XXX.XXX.XXX.XXX. reprezinta adresa IP
//    atribuita in pasul 1)
//
// 6. Se va afisa continutul paginii HTML editate pe ecran
//
#include "mn_userconst.h"           // librerie constate TCP-IP

```

```

#include "mn_stackconst.h"           // librerie constate TCP-IP
#include "mn_errs.h"                 // librerie coduri erori
#include "mn_defs.h"                 // librare definire tipuri
#include "mn_funcs.h"                // librerie prototipuri functii
#include "pagina\index.h"
#include <c8051F120.h>                // definitii specifice C8051F120

```

```

//-----
// Prototipuri functi
//-----

```

```

// Subrutine de initializare
void PORT_Init (void);
void SYSCLK_Init (void);
void EMIF_Init(void);
int establish_network_connection();

```

```

//-----
// Functia principala
//-----

```

```

void main(void)
{

```

```

    // dezactivare watchdog timer
    WDTCN = 0xde;
    WDTCN = 0xad;

```

```

    // initializare microcontroler
    PORT_Init();
    SYSCLK_Init();

```

```

    EMIF_Init();

```

```

    while(1)
    {
        // initializare stiva TCP-IP
        if (mn_init() < 0)
        {
            // daca ajunge in aceasta bucla inseamna ca ete eroare la initializarea stivei.
            // de verificat conexiunea cablurilor si a placii
            while(1);
        }
    }

```

```

    // conectarea la retea
    establish_network_connection();

```

```

// adaugare pagina web la sistemul virtual de fisiere al serverului
// pagina principala trebuie denumita index.htm sau index.html
mn_vf_set_entry((byte *)"index.html", INDEX_SIZE, index_html,
VF_PTYPE_FLASH);

// porneste serviciile de retea nivelul aplicatie
mn_server();

}
}

//-----
// stabilire conexiune retea
//-----
//
// functia apeleaza mn_ether_init() pentru initializarea mechetei si conectarea
// la retea.
//
// daca se efeteuaza coenxiunea la retea functia returneaza 1
//
// daca nu exista nici o conexiune functia asteapta pana apare o coenxiune
// sau macheta este resetata in acest moment se pot efectua si alte task-uri
//
int establish_network_connection()
{
    int retval;

    do
    {
        // mn_ether_init() initializeaza controlerul Ethernet.
        // AUTO_NEG indica faptul ca controlerul ur demareaza auto-negocierea
        retval = mn_ether_init(AUTO_NEG, 3, 0);

        // daca nu apare nici o legatura interogheaza link_status pana la reset
        // sau stabilirea unei conexiuni
        if (retval == LINK_FAIL)
        {
            while(!link_status && !ether_reset);
        }

        // daca retval este mai mic ca 0 sau e diferit de AUTO_NEG_FAIL , eroare conexiune

        else if ((retval < 0) && (retval != AUTO_NEG_FAIL))
        {
            // verifica daca controlerul ethernet este alimentat si conectat corect.

```

```

        // verifica daca macheta este configurata la viteza compatibila controlerului

        while(1);
    }

}while((retval < 0) && (retval != AUTO_NEG_FAIL));

return (1);

}

```

```

//-----
// subrutine de initializare
//-----

//-----
// PORT_Init
//-----
//
// configureaza intreruperile , alocarea pinilor si porturile paralele
//
void PORT_Init (void)
{
    char SFRPAGE_SAVE = SFRPAGE;    // salvare SFR curenta

    SFRPAGE = CONFIG_PAGE;          // setare pagina SFR
    P1MDOUT |= 0x40;                  // setare P1.6(TB_LED)
    P2MDOUT |= 0x08;                  // Setare P2.2(AB4_LED1)
    // toti pinii magistrala externa pusi in modul push-pull
    P4MDOUT = 0xDF;
    P5MDOUT = 0xFF;
    P6MDOUT = 0xFF;
    P7MDOUT = 0xFF;
    P4 = 0xC0;                        // /WR, /RD, 1 RESET 0
    P5 = 0xFF;
    P6 = 0xFF;                        // P5, P6 linii de adrese
    P7 = 0xFF;                        // P7 linii de date

    TCON &= ~0x01;                    // INT0 apare la triggerare

    // Validare UART0, CP0, ;i /INT0. /INT0 pe0.3.
    XBR0 = 0x84;
    XBR1 = 0x04;
    XBR2 = 0x40;

```

```

    SFRPAGE = SFRPAGE_SAVE;    // restaurare pagina SFR

}
//-----
// EMIF_Init
//-----
//
// configureaza memoria externa
//
void EMIF_Init (void)
{
    char SFRPAGE_SAVE = SFRPAGE; // salvare pagina SFR curenta

    SFRPAGE = LEGACY_PAGE;

    EMIOCF = 0xFB;           // Split-mode, non-multiplexed pe P0 - P3

    EMIOTC = EMIF_TIMING;    // constata modificata in functie de SYSCLK

    EMIOCN = BASE_ADDRESS;   // pagina din XRAM accesata de EMIF
    SFRPAGE = SFRPAGE_SAVE;  // restaurare pagina SFR

}

//-----
// SYSCLK_Init
//-----
//
// initializeaza ceasul sistemului
//
void SYSCLK_Init (void)
{
    int i;                   // software timer

    char SFRPAGE_SAVE = SFRPAGE; // salvarea pagina SFR

    SFRPAGE = CONFIG_PAGE;    // setare pagina SFR

    OSCICN = 0x83;           // setare oscilator intern la frecventa maxima

    CLKSEL = 0x00;           // selecteaza oscilator intern la sursa SYSCLK

    //porneste bucla PLL si creste ceasul cu factorul M/N
    PLL0CN = 0x00;           // setare oscilator intern la PLL
    SFRPAGE = LEGACY_PAGE;

```

```

    FLSCCL = 0x30;          // seteaza ceasul de citire flash la 100MHz
    SFRPAGE = CONFIG_PAGE;

    PLL0CN |= 0x01;          // alimentare bucla PLL

    PLL0DIV = 0x01;          // setare valoare predivizor N (N = 1)
    PLL0MUL = 0x03;          // inmultire SYSCLK cu M (M=3)
    PLL0FLT = 0x01;          // setare PLL pentru referinta la
                             // 12.2 - 19.5 MHz si iesire la 65 - 100 MHz
    for (i=0; i < 256; i++) ; // temporizare
    PLL0CN |= 0x02;          // validare PLL
    while(!(PLL0CN & 0x10)); // asteapta pana frecventa PLL e obtinuta
    CLKSEL = 0x02;          // selecteaza PLL ca sursa SYSCLK

    SFRPAGE = SFRPAGE_SAVE; // restaureaza pagina SFR
}

//-----
// ether_reset_low
//-----
//
// seteaza pinul de reset la controler pe 0 logic.
//
void ether_reset_low()
{

    unsigned char SFRPAGE_Save;

    SFRPAGE_Save = SFRPAGE; // salvare pagina SFR curenta
    SFRPAGE = CONFIG_PAGE;  // comuta pe pagina config SFR

    P4 &= ~0x20;            // reset 0 logic

    SFRPAGE = SFRPAGE_Save; // restaurare pagina SFR
}

//-----
// ether_reset_high
//-----
//
// Functia plaseaza pinul de reset in High-Z ceea ce permite sa fie setat de
// un buton utilizand un rezistor pull-up.
//
//asteapta citirea pinului de reset inainte de a se iesi din functie.

```

```
//
void ether_reset_high (void)
{

    unsigned char SFRPAGE_Save;

    SFRPAGE_Save = SFRPAGE; // salvare pagina SFR curenta
    SFRPAGE = CONFIG_PAGE; // comuta pe pagina SFR porturi

    P4 |= 0x20; // setare reset
    while(!(P4 & 0x20)); // asteapta actionare reset

    SFRPAGE = SFRPAGE_Save; // restaurare pagina

}

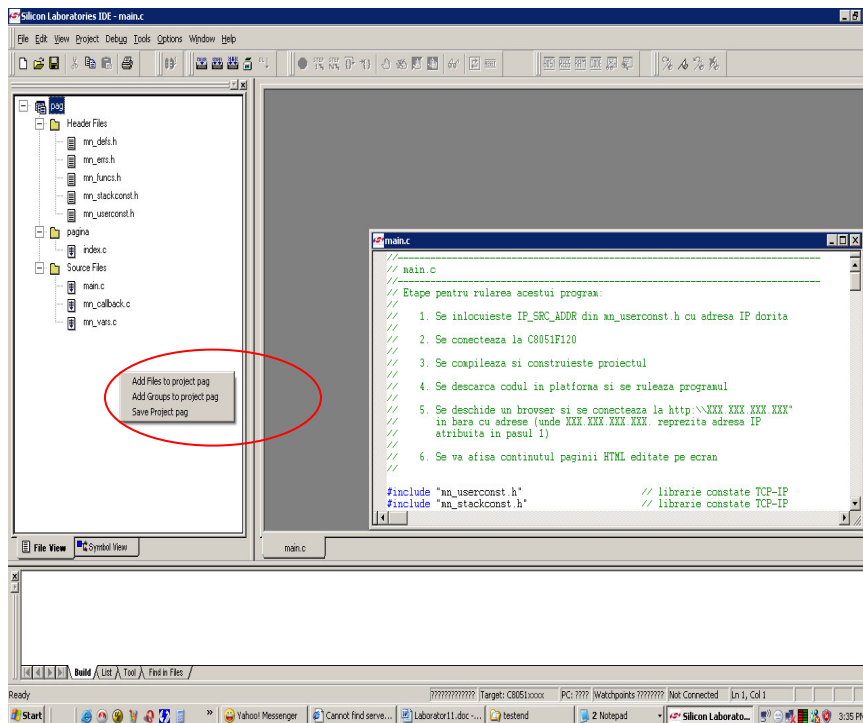
```

Se va salva.

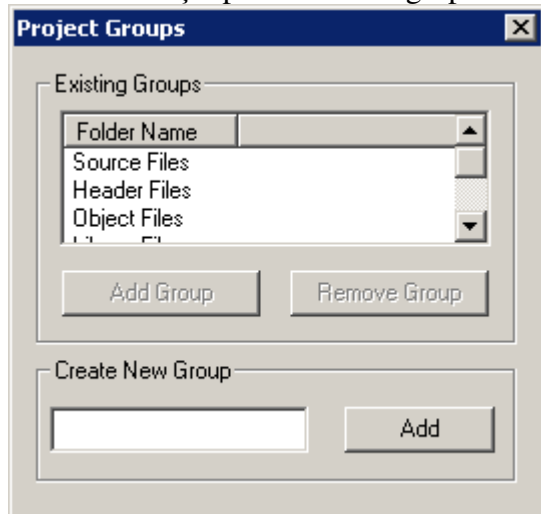
Se vor vrea în proiect următoarele grupuri de fișiere:

- header files;
- source files;
- pagina.

Pentru crearea unui grup se va da click dreapta pe spațiul alocat administrării proiectelor în SilabsIDE. Din meniul care apare se selectează Add groups to project pag.



Va apărea o fereastră ca mai jos. Din Existing groups se vor adauga Source Files și Header Files și apoi se va crea grupul nou numit pagina care se va adauga.



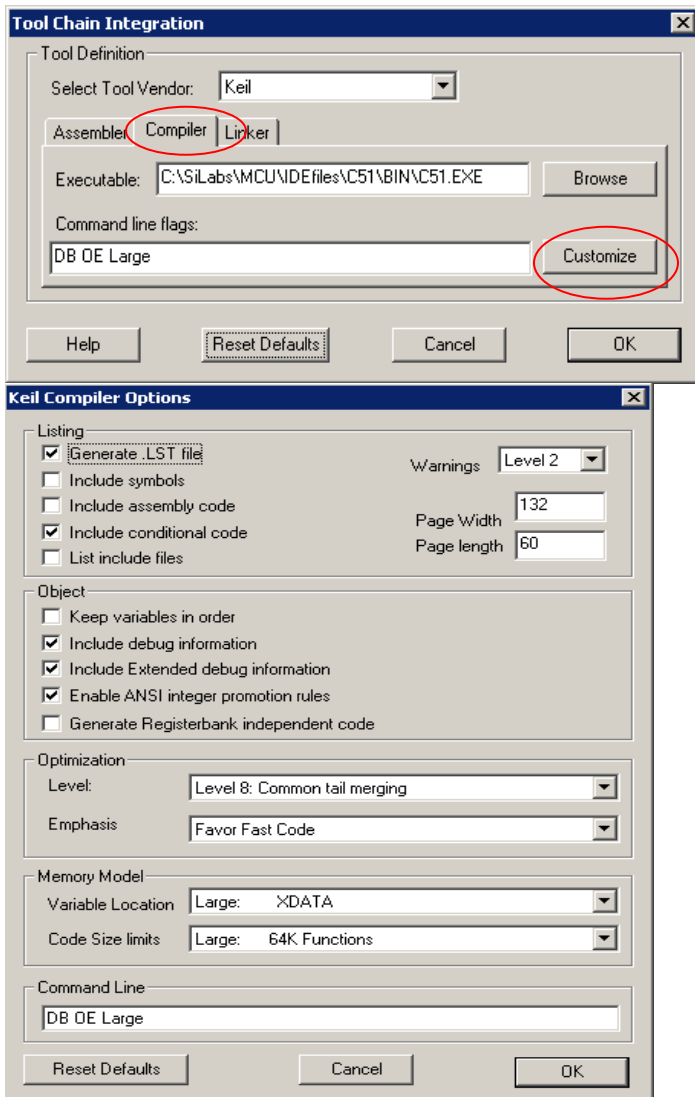
Pentru adăugarea unui fișier într-un grup se va da click dreapta pe grupul respectiv și apoi se va selecta Add file to group.

Se vor adăuga următoarele fișiere în următoarele grupuri:

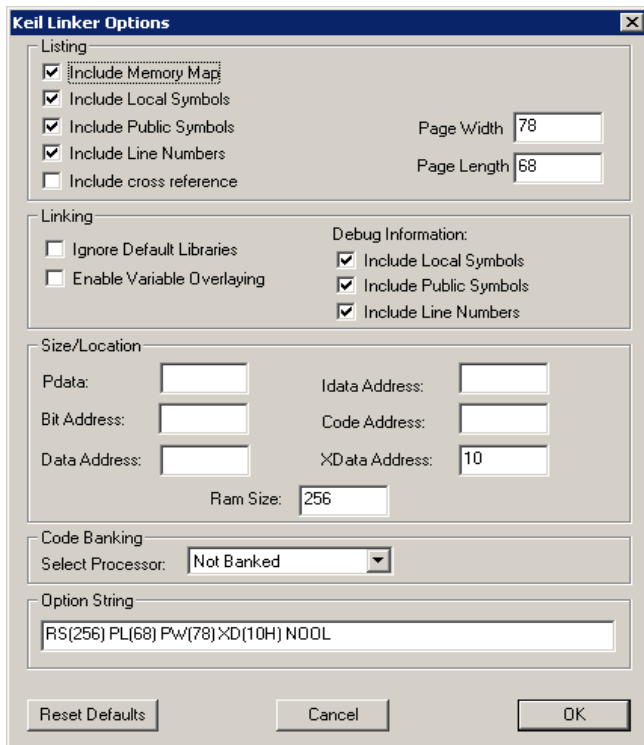
Grup	Fișiere
Header Files	Mn_defs.h
	Mn_errs.h
	Mn_funcs.h
	Mn_stackconst.h
	Mn_userconst.h
Source Files	Main.c
	Mn_callback.c
	Mn_vars.c
Pagina	Index.c

Se vor face următoarele setări la proiect.

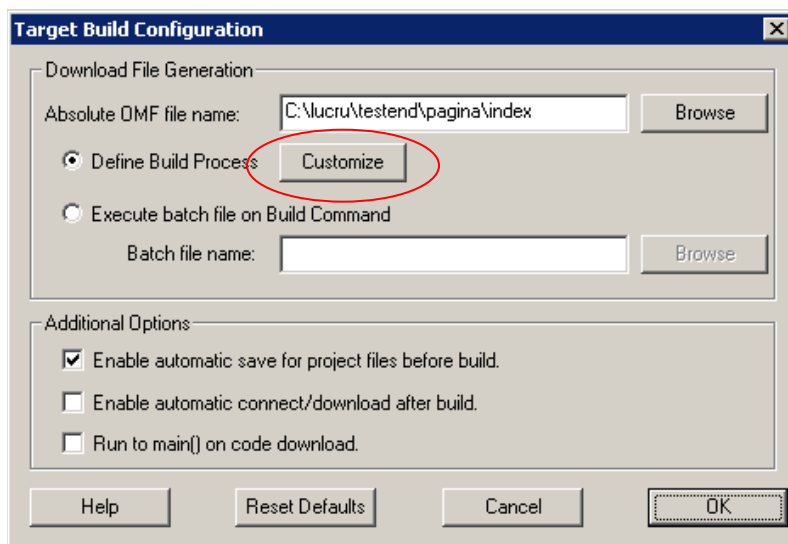
Din meniul Project – Tool chain integration apoi se va selecta tag-ul Compiler și se va apăsa butonul Customize. Se vor face setările ca în fereastra de mai jos:



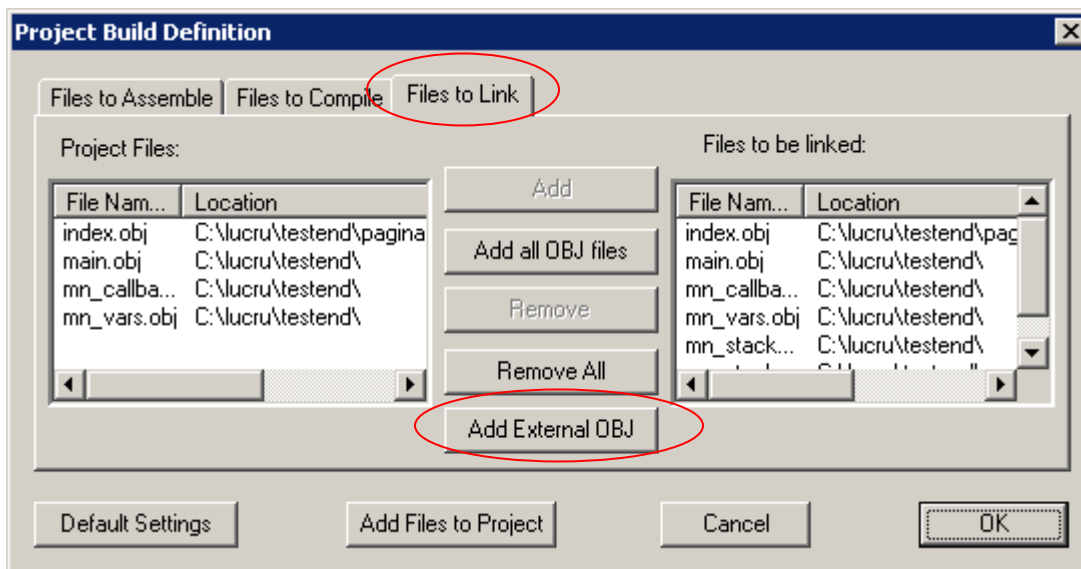
Se va da OK se va selecta apoi tag-ul linker și se va apăsa butonul Customize. Se vor face setările ca în fereastra de mai jos:



Din meniul Project – Target Build Configuration apoi se va acționa butonul Customize



Din fereastra care apare alegem:



Se vor adăuga fișierele librărie

Mn_stack_bank_039.lib
Mn_stack_common_039.lib

Se va compila și construi întreg proiectul. Dacă toate etapele au fost parcurse corect trebuie să apară 0 erori și 0 atenționări. Dacă apar erori sau atenționări înseamnă că anumite etape din cele de mai sus au fost sărite sau au fost executate incorect.

Dacă compilarea funcționează se va conecta la machetă și se va descărca fișierul obiect. Se va rula aplicația.

Rularea aplicației

Pentru a putea vedea ceva funcțional macheta va trebui să aibă conectată interfața ethernet ușor de recunoscut prin mufa specifică. Se va conecta cablul ethernet între machetă și calculator. Pentru a putea funcționa va trebui ca macheta și calculatorul să aibă adrese IP din aceeași clasă.

Setările la machetă sunt următoarele (se pot schimba – vedeți indicațiile din fișierul main):

Adresa IP	192.168.96.59
Masca rețea	255.255.255.0
Adresa gateway	192.168.96.1

Calculatorul trebuie configurat în aceeași clasă de adrese, dar nu să aibă aceeași adresă. Propunem de exemplu următoarea variantă.

Adresa IP	192.168.96.50
Masca rețea	255.255.255.0
Adresa gateway	192.168.96.1

Se va deschide un browser internet. Se va tasta în câmpul de adrese:

<http://192.168.96.59>

În mod normal pe ecran va trebui să apară fereastra care a fost creată.

Aplicații:

1. Se vor modifica tag-urile HTML. Se va crea o a doua pagină și se va face legătura cu ea prin intermediul TAG-ului `<a href=> </a>`
2. Se va adăuga o imagine în pagină prin tag-ul `<img src=>`.
3. Există vreo posibilitate ca o variabilă care contorizează numărul de impulsuri care vin pe un pin de port să fie afișată în pagină?